guard-forbidden-commands.sh

Revision: 2 **Last modified:** 2026-08-19T00:00:00Z

Rev 2 change (BOB-099 / 2026-08-19) — closed the CARRIER-vs-INVOCATION false-positive class (§11.4.196(D) / §11.4.201(7)(a)) for the emulator, force-push, `--no-verify`, and `--no-gpg-sign` gates. Prior to this fix, those four gates scanned raw `$COMMAND` with word-boundary regexes, so a harmless carrier like `echo 'emulator -avd is dev-only'`, `ls -la # git push --force is banned`, or a subagent prompt string that merely quoted the trigger tokens was BLOCKED as if the shell were actually going to run them. The sudo/su + host-power gates already routed through the quote-/comment-/ heredoc-aware `SCRUBBED_COMMAND` projection; the fix HOISTS that projection above the emulator gate so every structural-match gate consumes the same scrubbed view, then switches the five affected regex sites from `$COMMAND` to `$SCRUBBED_COMMAND`. The escape-hatch marker check (`# guardrails:allow <reason>`) deliberately stays on raw `$COMMAND` — after scrubbing, the marker's own comment would be replaced by filler and vanish. Hermetic coverage added at `scripts/hooks/test_guard_forbidden_commands.sh` (32 cases: 11 golden-TRUE real invocations, 16 golden-FALSE carrier fixtures across every gate class, 1 non-Bash tool passthrough, 2 escape-hatch downgrade cases, 2 host-power no-override cases — the whole set exits 0). No true-positive class was weakened. **Authority:** constitution §11.4.109 (Mandatory Anti-Forgetting Enforcement: PreToolUse Guard Hook + Subagent Constitutional Preamble + Orchestrator Pre-Action Checklist) **Maintainer:** constitution submodule (inherited by reference per §11.4.177 / §11.4.28(B)) **Scope:** §11.4.18 companion doc for `constitution/scripts/hooks/guard-forbidden-commands.sh`

Overview

`guard-forbidden-commands.sh` is a Claude Code **PreToolUse guard hook** — the "anti-forgetting" mechanical enforcement floor of §11.4.109. It is the general forbidden-command counterpart to the more specific `guard-work-track-`

`binding.sh` (§11.4.191) and `guard-track-branch-label.sh` (§11.4.182) hooks: instead of a per-work-item or per-label check, it blocks four whole CLASSES of Bash invocation at the tool-call boundary, independent of any agent's memory of the constitutional rule.

Forensic origin (per the script's own header): during an earlier on-device-API build, emulator subagents ran raw host-direct `emulator` / `adb` instead of routing through the sanctioned Containers submodule path, because the orchestrator forgot to paste that rule into their dispatch prompt. A rule an orchestrator forgets to inject is not enforcement — this hook makes the forbidden/gated classes mechanical instead of memory-dependent.

The hook is deliberately **generic and portable**: it carries no ATMOSphere-specific paths, works with or without `jq` on `PATH` (falls back to a small embedded awk-based JSON-field extractor for exactly two fields), and is designed to be consumed by reference from the constitution submodule, never copied into a project tree (§11.4.109 / §11.4.177).

Prerequisites

- `bash` (the script is `#!/usr/bin/env bash` and uses bash-only constructs — `[[ ]]` extended test, `local -a` arrays, `BASH_REMATCH`, C-style `((...))` arithmetic. It is NOT POSIX-`sh` portable; `.claude/settings.json` invokes it explicitly as `bash "$CLAUDE_PROJECT_DIR/constitution/scripts/hooks/guard-forbidden-commands.sh"`, never via a bare `sh` shebang execution.)
- `jq` is preferred but OPTIONAL — when absent, the script's own `json_field()` awk fallback extracts `.tool_name` and `.tool_input.command` from the raw JSON payload (a small hand-rolled state machine that locates the key, skips to the following quoted string, and unescapes `\" \\ \n \t \r \/`). No other JSON path is supported by the fallback; if `jq` is present it is used unconditionally for both fields via `jq -r "$path // empty"`.
- Wired as a `PreToolUse` hook under the `Bash` matcher in `.claude/settings.json` (confirmed in this checkout, alongside `guard-branch-consistency.sh` and `guard-work-track-binding.sh` on the same matcher).

Usage

It is not run by hand — the Claude Code runtime invokes it for every `Bash` tool call, passing the full PreToolUse JSON payload on stdin. Exit code `0` allows the command; exit code `2` BLOCKS it, and everything the script wrote to stderr is fed back to Claude as the refusal reason. Per the script's own top-of-file contract comment, any exit code other than 0 or 2 would be "non-blocking error" — but the script never emits one; every path terminates via either `exit 0` (fall-through allow) or `block()`'s `exit 2`.

Manual smoke (mirrors the payload shape the runtime sends):

```
echo '{"tool_name":"Bash","tool_input":{"command":"git push --force origin main"}}' \
  | bash constitution/scripts/hooks/guard-forbidden-commands.sh ; echo "exit=$?"
# → stderr: "guardrails: BLOCKED — §6.T.3 force-push" + the refusal message; exit=2

echo '{"tool_name":"Bash","tool_input":{"command":"ls -la"}}' \
  | bash constitution/scripts/hooks/guard-forbidden-commands.sh ; echo "exit=$?"
# → exit=0, no output
```

Only `tool_name == "Bash"` is inspected (`if [[ "$TOOL_NAME" != "Bash" ]]; then exit 0; fi` , lines 114–116) — every other tool call (Read, Edit, Agent/Task dispatch, MCP tool calls, etc.) passes through untouched at exit 0 without the command body ever being read. An empty/missing `.tool_input.command` also allows immediately (lines 118–121).

Scope / forbidden classes (four, exactly as coded)

1. Emulator / device gate (§6.X / §6.V / §6.AG) — lines 151–176. Blocks:

- `emulator -avd ...` as a standalone word (not a substring of a longer token), via `(^|[^[:alnum:]_/.-])emulator[[:space:]]+-avd([[:space:]]|$)` .
- A path ending in `/emulator` reached through an `$ANDROID_*` / `${ANDROID_*}` environment-variable reference (`\$(ANDROID_[A-Z_]+|\{ANDROID_[A-Z_]+\})[^[:space:]]*/emulator([[:space:]]|$)`).
- `adb install` or `adb -s <serial> install` at top level.

- `am instrument` (raw instrumentation-runner invocation).

All four hits share one message (`EMULATOR_MSG`): gate emulator runs MUST go via `scripts/run-challenge-matrix.sh` → the Containers submodule; raw host-direct `adb` / `emulator` is dev-iteration only, never gate evidence.

2. Force-push / verification-bypass gate (§6.T.3) — lines 177–299. Blocks `git ... push` invocations carrying `--force` , `-f` , `--force-with-lease` , or a `+<refspec>` force-push marker, PLUS bare `--no-verify` / `--no-gpg-sign` anywhere in the command. Detection mechanics are covered in detail below — this is the section the 2026-07-11 false-positive fix landed in.

3. sudo / su gate (§6.U) — lines 300–316. Blocks a standalone `sudo` token and a standalone `su` token (any arguments after `su` are blocked too — e.g. `su root -c '<anything>'` — per a documented forensic fix (`F3-B1`) that closed a bypass where the previous regex only recognised `su -` , `su -l` , or bare `su` , letting `su <user> -c ...` slip through). Both checks recognise `;` , `|` , `&` as valid preceding-boundary characters in addition to whitespace/start-of-string, since (unlike Section 2) this section does not use the clause splitter and instead scans the whole raw `$COMMAND` string directly.

4. Host-power gate (Host Machine Stability Directive) — lines 317–334. Blocks `systemctl suspend|hibernate|hybrid-sleep|suspend-then-hibernate|poweroff|halt|reboot|kexec|kill-user|kill-session` , `loginctl suspend|hibernate|hybrid-sleep|suspend-then-hibernate|poweroff|halt|reboot|kill-user|kill-session|terminate-user|terminate-session` , `pm-suspend|pm-hibernate|pm-suspend-hybrid` , and bare `shutdown` . This is the ONLY class passed `no-override` as `block()` 's third argument — it is categorically forbidden even with a documented `# guardrails:allow` marker, because destroying an in-progress build/session has no in-band justification.

Behaviour matrix

Situation	Result
`tool_name` ≠ `"Bash"`	allow (exit 0) — command body never inspected
`.tool_input.command` empty/absent	allow (exit 0)
Command matches one of the 4 forbidden classes above, no `# guardrails:allow` marker anywhere in the command	**BLOCK (exit 2)**, refusal text on stderr
Command matches a class ≠ host-power AND the command contains `# guardrails:allow <reason>` (reason non-empty) anywhere in the raw string	**WARN** (stderr) + **allow (exit 0)** — one marker in the command covers every non-power finding checked afterward in the same invocation (the marker flag is computed once, globally, before any class check runs)
Command matches a host-power class, WITH or WITHOUT a `# guardrails:allow` marker	**BLOCK (exit 2)**, no-override — the marker is acknowledged in the stderr text ("this class is NOT overridable") but never downgrades the block
`# guardrails:allow` present with no reason text after it (`(.+)` requires ≥1 char)	marker regex does NOT match → `ALLOW_MARKER_PRESENT` stays 0 → every hit BLOCKs as usual
No class matched (and/or only WARN-level findings fired)	allow (exit 0) — final fallthrough

The force-push detection specifics (- - fix)

The script's own header comment (lines 180–208) documents the forensic reproduction and fix in detail; this section restates it as the doc's authoritative summary, cross-checked line-by-line against the current code.

The bug (fixed). A benign command referencing a §11.4.88 push-failure log path (e.g. `git fetch --all && tail -f qa-results/push_failures/x.log`), or any command chaining a `git` invocation with an unrelated `-f`-flagged command whose OWN argument merely started with the literal characters `push`, was wrongly BLOCKED as a force-push. Two independent over-matches in the prior single whole-command regex:

- (a) the "is this a git-push invocation" test matched literal `push` as an **unbounded prefix** with no trailing word boundary — so `push_failures/...`, `push_all.sh`, or a bare "push" inside unrelated prose all satisfied it;
- (b) the force-flag test (`-f` / `--force` / `--force-with-lease`) scanned the **entire raw command string** with no notion of "clause" — so a `-f` flag belonging to a completely unrelated command chained via `&&` / `;` / `|` (the extremely common `tail -f <logfile>`) satisfied it even though it had nothing to do with any `git push`.

The fix (current code).

1. **Word-bounded git-push detector** — `GIT_PUSH_RE` (line 279): `'(^|[[:space:]])git([[:space:]]+[^[:space:]]+)*[[:space:]]+push([[:space:]]|$)'`. Both `git` and `push` must be complete, boundary-anchored words; `push` can never match as a mere token prefix.
2. **Quote-aware clause splitting** — `_fp_split_clauses()` (lines 218–275) is a pure-bash character-by-character state machine that splits the raw command on `;`, `&&`, `||`, a single (non-doubled) `|`, and literal newlines — **except** when those characters occur inside a single- or double-quoted string, in which case they are copied through untouched (double-quote handling additionally respects a backslash-escaped `\"` via the `prev` character check). A real `git push -o ci.skip="a;b" --force` is therefore never sliced apart at the `;` inside the quoted `-o` value.

3. **Same-clause requirement** — the script iterates the split clauses (`while IFS= read -r fp_clause; do ... done < <(_fp_split_clauses "$COMMAND")` , lines 281–291); a clause is only escalated to `block()` if it BOTH matches `GIT_PUSH_RE` AND, within that SAME clause, matches `--force` (word-bounded, optional trailing `=`), `-f` (standalone flag word), `--force-with-lease` (word-bounded, optional `=`), or a leading `+` refspec marker (`(^|[[:space:]])\+[^[:space:]]`).
4. **Malformed input can only merge, never split, clauses** — an unterminated quote just swallows the remainder of the string into one clause (only *unquoted* separators split), so a genuine `git push ... --force` can never be accidentally sliced apart from its force flag by adversarial or malformed input; §11.4.113's "force-push has no escape hatch" guarantee is preserved by construction.

Not clause-scoped. `--no-verify` and `--no-gpg-sign` (lines 293–298) are checked with `[[ "$COMMAND" =~ (^|[[:space:]])--no-verify([[:space:]]|$) ]]` and the `--no-gpg-sign` equivalent — these two scan the **entire raw `$COMMAND` string directly**, NOT the clause-split output, and are NOT conditioned on being part of a `git push` invocation at all. A `--no-verify` token anywhere in a Bash tool call (any command, any position, any clause) blocks the call.

Edge cases

- **Quoted prose is not excluded outside Section 2.** Sections 1 (emulator), 3 (sudo/su), and 4 (host-power) test the whole raw `$COMMAND` string with `[[ … =~ … ]]` directly — they do not use `_fp_split_clauses()` and are not quote-aware. A command whose ONLY reference to a forbidden token is inside a quoted string (e.g. `echo "please don't sudo this"` , or a commit message argument that happens to quote the words `am instrument`) can still satisfy the word-boundary regex and trigger a BLOCK/WARN for that class, because the regex has no notion of "inside quotes" for these four sections. This is the current, code-confirmed behaviour — not a hypothesis.
- **`--no-verify` / `--no-gpg-sign` are whole-command, not git-scoped.** As noted above, these two checks are independent of the `git push` detector and independent of clause boundaries; they fire on the raw string.

- **One marker covers the whole invocation.** `ALLOW_MARKER_PRESENT` / `ALLOW_REASON` are computed ONCE from the full command (lines 127–132) before any of the four sections run; `block()` re-reads this same global flag on every call. A single `# guardrails:allow <reason>` anywhere in a multi-clause command downgrades EVERY non-host-power finding in that same command to a WARN, not just the finding nearest the marker.
- **Reason capture is greedy to end-of-string.** The marker regex's capture group is `(.+)` with no trailing anchor other than the implicit end of the matched string, so `BASH_REMATCH[1]` captures everything after `guardrails:allow` to the end of the command (including any further shell syntax that happens to follow it on the same line).
- **`am instrument` / `adb install` are unconditional inside Bash calls.** There is no allowlist for a "safe" instrumentation target — any occurrence of the word-bounded pattern blocks, escape-hatch-overridable only via the documented marker (unlike host-power).

Honest boundary (§ . .)

This is best-effort, mechanical prevention at the tool-call boundary — it is NOT a security boundary and does not claim to be. It only inspects the `Bash` tool's `command` string as received by Claude Code; it cannot see inside a script the command merely invokes (e.g. `bash some_wrapper.sh` where `some_wrapper.sh` itself runs `git push --force` — that would need the wrapper's OWN source to be caught by a different mechanism, such as a pre-commit/pre-push scan of tracked scripts, per §11.4.113's recommended `CM-NO-FORCE-PUSH-ABSOLUTE` gate). It also does not attempt semantic understanding of shell quoting beyond the deliberate clause-splitter added for Section 2 — Sections 1/3/4 remain raw-string regex checks as documented above under "Edge cases." The escape hatch (`# guardrails:allow <reason>`) is an audited WARN, not a bypass: the reason text is mandatory and lands in the transcript, and it is categorically inapplicable to host-power commands.

Related scripts

- `guard-work-track-binding.sh` (§11.4.191) — the work-to-track/branch placement guard, wired on the same `Bash` matcher.
- `guard-branch-consistency.sh` (§11.4.181) — the create-time feature-branch naming guard, wired on the same `Bash` matcher.
- `guard-track-branch-label.sh` (§11.4.182) — the `(T<N>/<branch>)` label guard, wired on the `Agent|Task|TaskCreate` matcher.
- `docs/AGENT_GUARDRAILS.md` (constitution submodule) — the §11.4.109 SUBAGENT CONSTITUTIONAL PREAMBLE + ORCHESTRATOR PRE-ACTION CHECKLIST document this hook is the mechanical floor for (hook = floor, preamble = ceiling, per §11.4.109's framing).

Testing

The hermetic hook test suite for this guard currently lives in the CONSUMER (parent) tree at `scripts/hooks/test_guard_forbidden_commands.sh` — **49 cases** (≥ 20 per §11.4.109): every blocked class exits 2, benign/allowed commands exit 0, the `# guardrails:allow <reason>` escape-hatch WARNs-not-blocks for non-power classes, and host-power rejects even with the marker. Independently re-run 2026-07-11 after the force-push false-positive fix landed: **49/49 PASS**, including `git push --force` / `--force-with-lease` / `-f` / `+refspec` still BLOCKED (§11.4.113 preserved) and the `qa-results/push_failures/…`, `push_all.sh`, and prose-mentioning-"push force" false-positives now correctly ALLOWED.

```
bash scripts/hooks/test_guard_forbidden_commands.sh
```

§11.4.28 / §11.4.109 placement note (tracked follow-up). Because this guard is inherited BY REFERENCE from the constitution submodule (§11.4.177), its hermetic test ideally travels WITH it — i.e. a constitution-side `constitution/scripts/hooks/test_guard_forbidden_commands.sh` — so every consuming project inherits the coverage, not only this project. The sibling guards' tests (`test_guard_branch_consistency.sh`, `test_guard_track_branch_label.sh`,

`test_guard_work_track_binding.sh`) already live constitution-side; this guard's does not yet. Relocating it (with the path adjustments the move requires) is a separate, trackable workable item.

Last verified: 2026-07-11 — `bash -n` clean; wiring confirmed present in `.claude/settings.json` under the `Bash` PreToolUse matcher; the parent `scripts/hooks/test_guard_forbidden_commands.sh` re-run 49/49 (conductor); every behaviour documented above was read directly from the script source (`constitution/scripts/hooks/guard-forbidden-commands.sh`, 340 lines).